

6강. 컴포넌트 스타일링



React

리엑트 CSS

▪ index.css

프로젝트 전체에 적용되는 기본 스타일 특징

- 모든 컴포넌트에 영향을 줌
- HTML 기본 스타일 초기화(reset) 또는 공통 스타일 정의
- 폰트, 배경, 기본 색상 설정

```
/* index.css */  
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  background-color: #f5f5f5;  
}  
  
h1 {  
  color: #333;  
}
```



리엑트 CSS

▪ App.css

특징

- 특정 컴포넌트에만 적용
- 보통 `App.jsx` UI 스타일 담당

```
/* App.css */  
.container {  
  text-align: center;  
  padding: 20px;  
}  
  
.button {  
  background-color: blue;  
  color: white;  
}
```



리엑트 CSS

▪ index.css vs App.css

구분	index.css	App.css
범위	전역(Global)	컴포넌트(Local 느낌)
적용 대상	전체 앱	특정 컴포넌트
import 위치	main.jsx	App.jsx
용도	초기화, 공통 스타일	UI 스타일

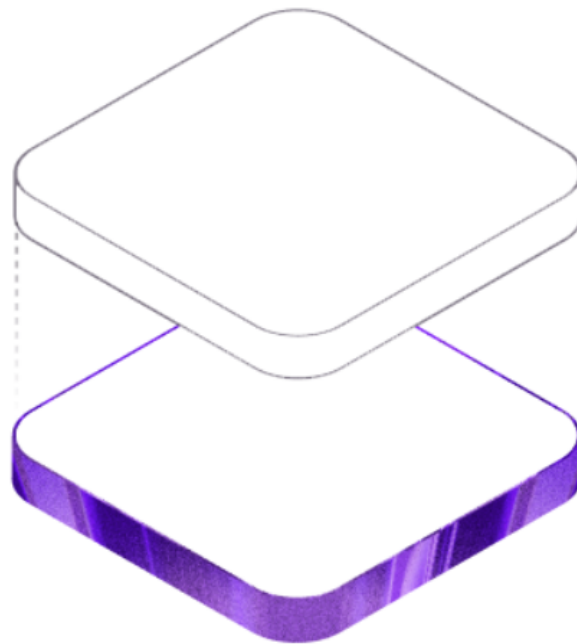


컴퓨터 기기 쇼핑몰

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

컴퓨터 기기 쇼핑몰

최신 컴퓨터 기기를 한 곳에서 만나보세요!



컴퓨터 기기 쇼핑몰

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

상품 리스트

상품ID	상품명	가격	설명
1	모니터	130000	22인치 최신 모니터입니다.
2	마우스	30000	무선 마우스입니다.
3	키보드	50000	무선 키보드입니다.

[상품 등록하기](#)



컴퓨터 기기 쇼핑몰

[Home](#)[상품 목록](#)[상품 등록](#)[로그인](#)

상품 정보

상품 ID: 2

상품 이름: 마우스

상품 가격: 30000

상품 설명: 무선 마우스입니다.



컴퓨터 기기 쇼핑몰

[Home](#) [상품 목록](#) [상품 등록](#) [로그인](#)

상품 등록

상품명

상품명을 입력하세요

가격

가격을 입력하세요

설명

상품 설명을 입력하세요

등록

취소



컴퓨터 기기 쇼핑몰

Home

상품 목록

상품 등록

로그인

로그인

ID 입력

패스워드 입력

로그인



컴퓨터 기기 쇼핑몰

▪ index.css

```
*{  
  margin: 0;  
  padding: 0;  
}  
body {  
  font-family: Arial, sans-serif;  
  color: ■#333;  
}  
a {  
  color: ■#333;  
  text-decoration: none;  
}  
a:hover {  
  text-decoration: underline;  
}  
h1, h2, h3 {  
  margin: 20px;  
  text-align: center;  
}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
/* Header 스타일 */  
.header{  
  width: 100%;  
  background-color: ■#333;  
  padding: 20px;  
  text-align: center;  
}  
.header a {margin: 0 16px; color: orange; text-decoration: none;}  
.header a:hover {text-decoration: underline;}  
.header-user {  
  display: inline-flex;  
  align-items: center;  
  gap: 10px;  
  color: □#fff;  
  margin-left: 12px;  
}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
.logout-btn {  
  padding: 6px 12px;  
  border: 1px solid #ff9f1c;  
  border-radius: 6px;  
  background-color: transparent;  
  color: #ff9f1c;  
}  
  
.logout-btn:hover {background-color: #ff9f1c; color: #333;}  
  
/* Main 스타일 */  
.main {  
  width: 1000px;  
  margin: 0 auto;  
  padding: 20px;  
  text-align: center;  
}  
  
.main img {margin-top: 20px;}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
/* ProductList 스타일 */
.product-list {
  width: 1000px;
  margin: 0 auto;
  text-align: left;
}
.table-list {
  width: 60%;
  margin: 0 auto;
  border-collapse: collapse;
}
.table-list th, .table-list td {
  border: 1px solid #ddd;
  padding: 8px;
}
.table-list th {text-align: center; background-color: #f2f2f2;}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
.table-list th {text-align: center; background-color: #f2f2f2;}
.btn-add {
  text-align: right;
  margin-top: 20px;
  margin-right: 20%;
}
.btn-add button {
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #007bff;
  color: white;
  border: none;
  cursor: pointer;
}
.btn-add button:hover {background-color: #0056b3;}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
/* 상품 정보 */
.product-details {
  width: 50%;
  margin: 0 auto;
  text-align: left;
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 8px;
  background-color: #f0f0f0;
}
.product-details p {margin-bottom: 10px;}

/* 상품 등록 폼 */
.add-form {width: 40%; margin: 0 auto; text-align: left;}
.add-form label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
.add-form input,
.add-form textarea {
  width: 100%;
  padding: 8px;
  margin-bottom: 16px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.add-form button {
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #28a745;
  color: white;
  border: none;
  cursor: pointer;
}
.add-form button:hover {background-color: #218838;}
.add-form button[type="submit"] {margin-right: 10px;}
.add-form button[type="button"] {background-color: #6c757d}
.add-form button[type="button"]:hover {background-color: #5a6268;}
```



컴퓨터 기기 쇼핑몰

▪ App.css

```
/* 로그인 폼 */
.signin-form {width: 30%; margin: 0 auto; text-align: left;}
.signin-form input {
  width: 100%;
  padding: 8px;
  margin-bottom: 16px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
.signin-form button {
  width: 100%;
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #007bff;
  color: white;
  border: none;
  cursor: pointer;
  margin-left: 3%;
}
.signin-form button:hover {background-color: #0056b3;}
```



미디어 쿼리

■ Media Query

- 접속하는 장치(미디어)에 따라 **특정한 CSS 스타일을 사용하도록 함**
- 반응형 웹디자인에서 가장 많이 사용하는 방식
(반응형 레이아웃 - 가변그리드 레이아웃, 플렉스박스 레이아웃 방식 등)
- 미디어 쿼리 구문

@media 미디어 유형 [and 조건] * [and 조건]

예) 미디어 유형이 'screen'이면서 최소 너비가 '600px'이고 최대 너비가 '1400px' 정의

```
<style>  
  @media screen and (min-width: 768px) and (max-width: 1400px){  
    ....  
  }  
</style>
```



미디어 쿼리

▪ Media Query

```
<title>미디어 쿼리 예제</title>
<style>
  @media screen and (max-width: 767px) {
    body{
      background-color: lightblue;
    }
  }

  @media screen and (min-width: 768px) {
    body{
      background-color: lightyellow;
    }
  }

  @media screen and (min-width: 1200px) {
    body{
      background-color: lightgreen;
    }
  }
}
```



미디어 쿼리

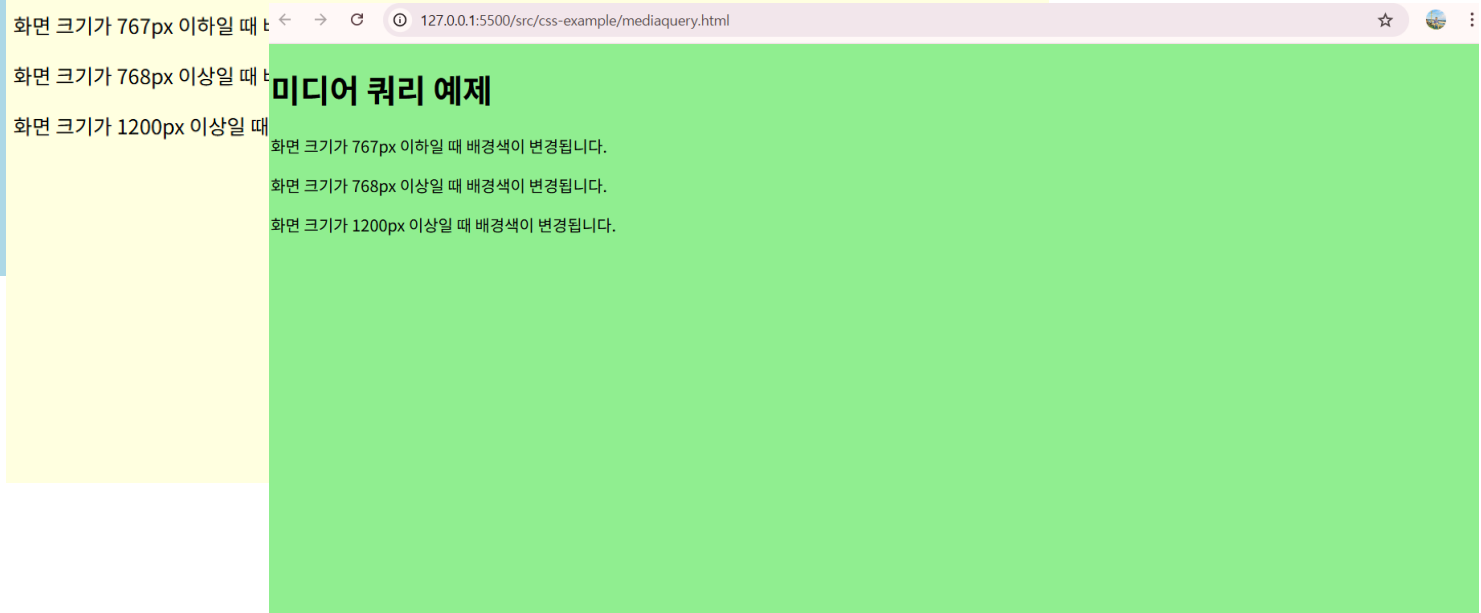
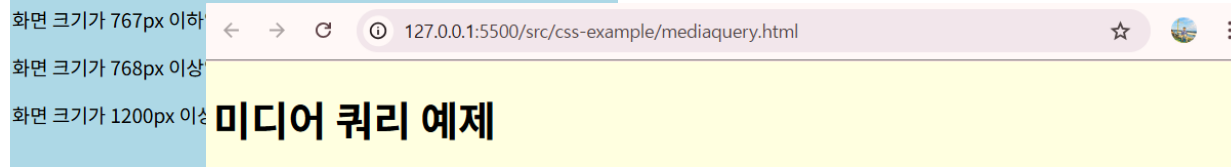
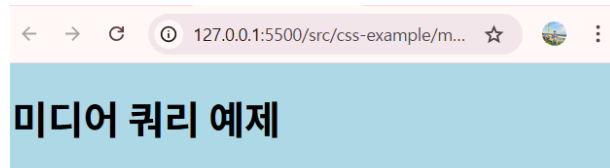
▪ Media Query

```
<body>  
  <h1>미디어 쿼리 예제</h1>  
  <p>화면 크기가 767px 이하일 때 배경색이 변경됩니다.</p>  
  <p>화면 크기가 768px 이상일 때 배경색이 변경됩니다.</p>  
  <p>화면 크기가 1200px 이상일 때 배경색이 변경됩니다.</p>  
</body>
```



미디어 쿼리

Media Query



인라인 스타일

▪ inline-style

- style 속성에 문자열이 아닌 자바스크립트 객체를 할당함.
(키는 카멜케이스 형태로 작성함)

```
return (  
  <>  
    <div>  
      { /* 인라인 스타일 적용 */}  
      <h1  
        style={{  
          color: 'red',  
          fontWeight: 'bold'  
        }}  
      >Hello, World!</h1>  
    </div>  
  </div>  
)
```



styled-components

- 리엑트 styled-components 란?

CSS를 JavaScript 안에서 작성할 수 있게 해주는 대표적인 **CSS-in-JS** 라이브러리입니다.

즉, 기존처럼 .css 파일을 따로 만드는 대신

JS 파일 안에서 스타일 + 컴포넌트를 함께 작성합니다.

- styled-components 설치

```
npm install styled-components
```

```
"dependencies": {  
  "axios": "^1.14.0",  
  "react": "^19.2.4",  
  "react-dom": "^19.2.4",  
  "styled-components": "^6.3.12"  
},
```



styled-components

■ 버튼 컴포넌트 스타일링

기본 버튼

녹색 버튼

보라 버튼

```
▼<div id="root">
  ▼<div class="App">
    <button class="sc-bpqTzb ejYZbX">기본 버튼</button> == $0
    <button color="green" class="sc-bpqTzb doGfUH" style="background-color:
    low;">녹색 버튼</button>
    <button class="sc-bpqTzb kvefPX">보라 버튼</button>
  </div>
</div>
<script type="module" src="/src/main.jsx?t=1774847964199"></script>
```



styled-components

■ 버튼 컴포넌트 스타일링

```
const ButtonSample = () => {  
  return (  
    <>  
      <Button>기본 버튼</Button>  
      { /* props를 이용하여 스타일을 동적으로 변경 */}  
      <Button color="green" backgroundColor="yellow">  
        |   녹색 버튼  
      </Button>  
      { /* primary prop이 true일 때 추가 스타일 적용 */}  
      <Button primary>보라 버튼</Button>  
    </>  
  );  
};  
  
export default ButtonSample;
```



styled-components

■ 버튼 컴포넌트 스타일링

```
import styled from 'styled-components';

const Button = styled.button`
  margin: 5px;
  padding: 8px 16px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 1rem;
  line-height: 1.5; // 버튼의 높이를 텍스트의 높이에 맞게 조정

  // props를 이용하여 스타일을 동적으로 변경
  color: ${props => props.color || 'black'};
  background-color: ${props => props.backgroundColor || 'white'};

  // primary prop이 true일 때 추가 스타일 적용
  ${props => props.primary && `
    color: white;
    background-color: purple;
  `}
`;
```



styled-components

- 동적 스타일링 문법

`${props => props.color}`는 styled-components의 동적 스타일링 문법입니다.

1. `${}` = JavaScript 템플릿 리터럴 (백틱 내에서 JS 실행)

2. `props => props.color` = 화살표 함수로 props 객체에서 color 값 추출

3. 결과 = 컴포넌트에 전달된 `color` prop을 CSS 값으로 사용



styled-components

- Main 페이지 스타일링

Hello~ React!

Click Me

Click Me



styled-components

- Main 페이지 스타일링

```
const MainPage = () => {  
  return (  
    <Wrapper>  
      <Title>Hello~ React!</Title>  
      <Button>Click Me</Button>  
      <RoundedButton>Click Me</RoundedButton>  
    </Wrapper>  
  );  
}  
  
export default MainPage;
```



styled-components

▪ Main 페이지 스타일링

```
import styled from 'styled-components';

const Wrapper = styled.div`
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  // 화면 전체 높이(viewport height) : 100 * 8 = 800px
  height: 100vh;
`;

const Title = styled.h1`
  font-size: 2.5rem;
  color: ■ #333;
  margin-bottom: 20px;
`;
```



styled-components

■ 컴포넌트 확장(상속)

```
const Button = styled.button`
  padding: 10px 20px;
  margin: 10px;
  font-size: 1rem;
  color: □ #fff;
  background-color: ■ #007bff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  &:hover {
    background-color: ■ #0056b3;
  }
`;

/* RoundedButton 컴포넌트는 Button 컴포넌트를
   상속받아 border-radius를 50px로 설정*/
const RoundedButton = styled(Button)`
  border-radius: 50px;
`;
```



뷰포트(Viewport) 단위

뷰포트(viewport)

- 실제 내용이 표시되는 영역
- PC화면과 모바일 화면의 픽셀 표시 방법이 다르기 때문에 모바일 화면에서 의도한 대로 표시되지 않음
- 뷰포트를 지정하면 기기 화면에 맞춰 확대/축소해서 내용 표시

뷰포트의 단위

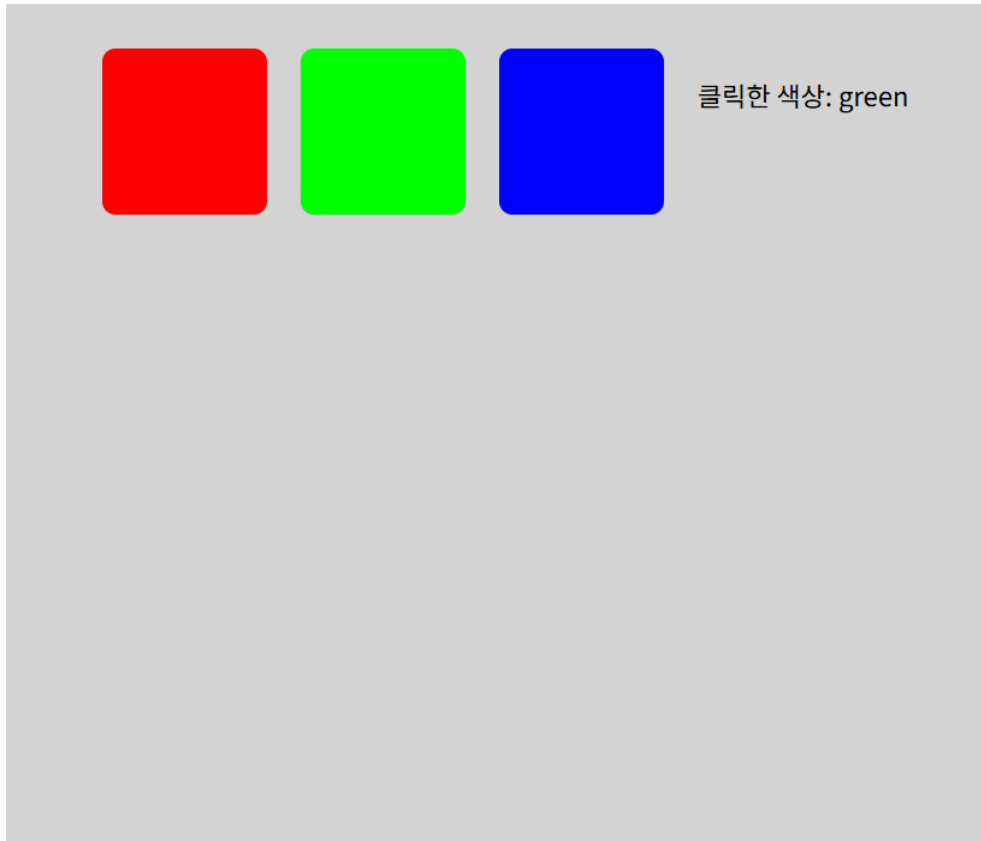
CSS에서 크기를 지정할 때 주로 px, % 단위는 고정 폰트이고, 브라우저의 비율에 따라 글자 크기가 변경되는 가변 폰트는 vw, vh이다.

- vw(viewport width) : 1vw는 뷰포트 너비의 1%와 같다.(1200px일때 5%-60px)
- vh(viewport height): 1vh는 뷰포트 높이의 1%와 같다.



styled-components

- Blocks 모형



styled-components

▪ Blocks 모형

```
const Blocks = () => {  
  // 클릭한 블록의 색상을 저장하는 상태  
  const [clicked, setClicked] = useState(null);  
  
  const handleClick = (color) => {  
    setClicked(color);  
  };  
  
  return (  
    <Wrapper>  
      <Block  
        color="■ #f00"  
        onClick={() => handleClick('red')}  
      />  
    )  
  );  
}
```



styled-components

▪ Blocks 모형

```
<Block
  color="#0f0"
  onClick={() => handleClick('green')}
/>
<Block
  color="#00f"
  onClick={() => handleClick('blue')}
/>
{clicked && <p>클릭한 색상: {clicked}</p>}
</Wrapper>
);
}

export default Blocks;
```



styled-components

▪ Blocks 모형

```
const Wrapper = styled.div`
  display: flex;
  flex-direction: row;
  align-items: flex-start;
  justify-content: center;
  gap: 20px;
  padding: 40px;
  background-color: lightgray;
  height: 100vh;
`;

const Block = styled.div`
  width: 100px;
  height: 100px;
  background-color: ${props => props.color};
  border-radius: 8px;
  cursor: pointer;
`;
```



일반 웹페이지 배포

■ 웹 페이지 배포

1. test web을 깃허브 저장소에 업로드 한다.

2. Settings > Pages 설정하기

1) **Settings** → **Pages**

2) **Build and deployment**:

- Source: ****Deploy from a branch****
- Branch: ****main****를 선택
- Folder: ****/ (root)**** 선택

3) **Save**



일반 웹페이지 배포

- Test 웹

홈페이지 방문을 환영합니다!!

오후 3:17:44



일반 웹페이지 배포

▪ index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Welcome~ Home</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>홈페이지 방문을 환영합니다!!</h1>
    <div id="demo" class="clock"></div>
  </div>

  <script src="clock.js"></script>
</body>
</html>
```



일반 웹페이지 배포

- style.css

```
.container {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}  
.clock {  
  font-size: 1.5em;  
  font-weight: bold;  
  color: ■ #00f;  
}
```



일반 웹페이지 배포

- clock.js

```
setInterval(() => {  
  let date = new Date();  
  let now = date.toLocaleTimeString();  
  document.getElementById("demo").innerText = now;  
}, 1000);
```



리엑트 앱 배포

- Github Pages에 배포 전 사전 작업

1. Product_mall 앱을 미리 깃허브 저장소에 업로드 한다.

2. Settings > Pages 설정하기

1) **Settings** → **Pages**

2) **Build and deployment**:

- Source: ****Deploy from a branch****
- Branch: ****main****를 선택
- Folder: ****/ (root)**** 선택

3) **Save**



리엑트 앱 배포

- Github Pages 에 배포

```
### 1.1 필수 패키지 설치
```bash
npm install
npm install gh-pages --save-dev
```
```



리엑트 앱 배포

▪ Github Pages 에 배포

```
## 📝 2단계: package.json 설정

### 필수 설정:
```json
{
 "name": "todo-app",
 "private": true,
 "version": "0.0.1",
 "type": "module",
 "homepage": "https://username.github.io/repository-name/",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "predeploy": "npm run build",
 "deploy": "gh-pages -d dist",
 "preview": "vite preview"
 },
}
```



# 리엑트 앱 배포

## ▪ Github Pages 에 배포

```
⚙ 3단계: vite.config.js 설정

필수 설정:
```javascript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  base: '/repository-name/', // GitHub Pages 경로
  server: {
    port: 5173
  }
})
```
```



# 리엑트 앱 배포

## ▪ Github Pages 에 배포

## 📌 4단계: index.html 설정

### 필수 설정 (상대 경로 사용):

```
```html
<!doctype html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>할 일 관리 앱</title>
  </head>
  <body>
    <div id="root"></div>
    <!-- ✅ 상대 경로 사용 (절대 경로 X) -->
    <script type="module" src="./src/main.jsx"></script>
  </body>
</html>
```
```



# 리엑트 앱 배포

## ▪ Github Pages 에 배포

## 🖥 5단계: 로컬 테스트

### 개발 서버 실행:

```
```bash
```

```
npm run dev
```

```
```
```

- <http://localhost:5173> 에서 접속
- 실시간 핫 리로드 제공

### 프로덕션 빌드 테스트:

```
```bash
```

```
npm run build
```

```
npm run preview
```

```
```
```

- ``dist/`` 폴더가 생성됨
- 빌드 결과를 테스트할 수 있음



# 리엑트 앱 배포

## ▪ Github Pages 에 배포

## 📁 7단계: 배포하기

### 7.1 배포 명령어:

```
```bash
npm run deploy
```
```

### 7.2 GitHub Pages 활성화:

1. GitHub 저장소 접속
2. **\*\*Settings\*\*** → **\*\*Pages\*\***
3. **\*\*Build and deployment\*\***:
  - Source: **\*\*Deploy from a branch\*\***
  - Branch: **\*\*gh-pages\*\***를 선택
  - Folder: **\*\*/ (root)\*\*** 선택 ☒ (gh-pages 브랜치의 루트 폴더)
4. Save





# 리엑트 앱 배포

## ▪ npm run deploy 후 > Published (배포됨)

```
PS D:\웹 배포\todo-app> npm run deploy

> todo-app2@0.0.1 predeploy
> npm run build

> todo-app2@0.0.1 build
> vite build

vite v4.5.14 building for production...
✓ 34 modules transformed.
dist/index.html 0.41 kB | gzip: 0.30 kB
dist/assets/index-d2ef3821.css 1.69 kB | gzip: 0.72 kB
dist/assets/index-fa09a36a.js 144.01 kB | gzip: 46.46 kB
✓ built in 880ms

> todo-app2@0.0.1 deploy
> gh-pages -d dist

Published
```



# 업데이트 후 배포

## ▪ Github Pages 에 배포

##  업데이트 시 배포

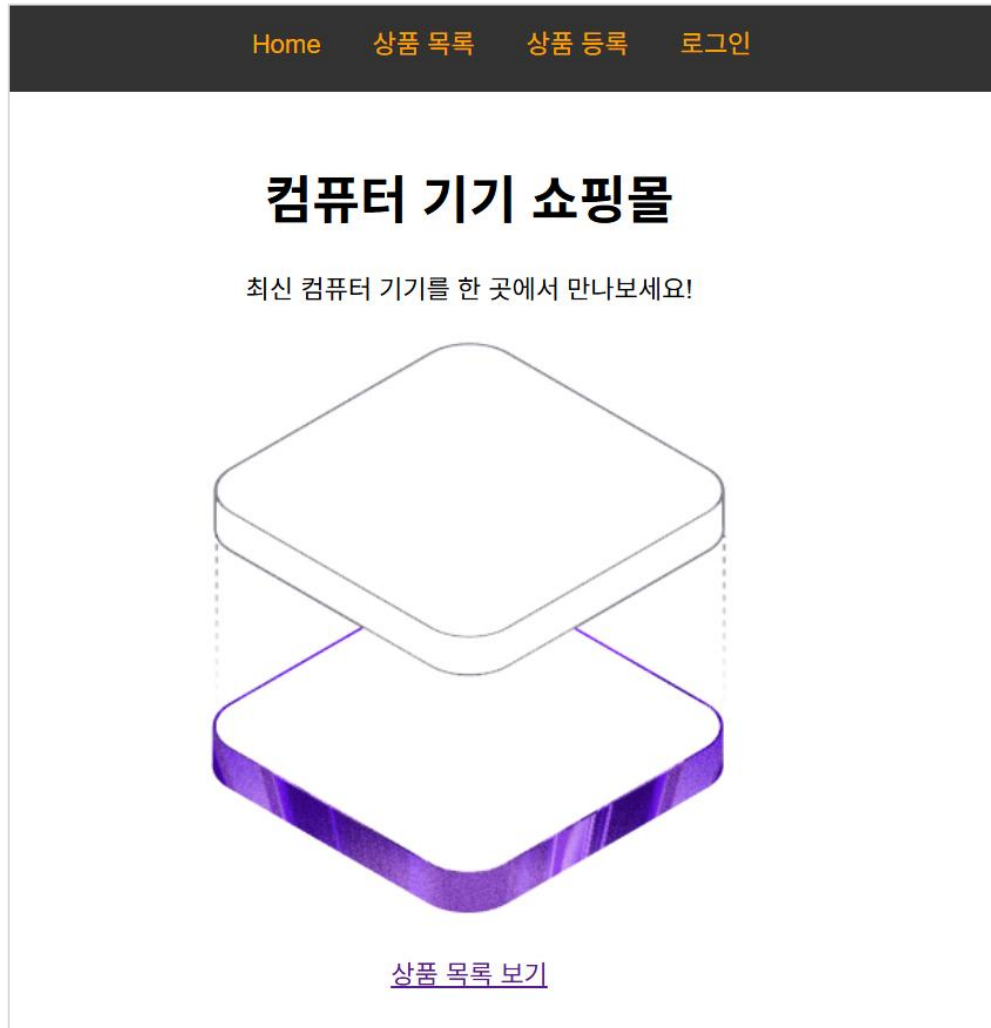
코드 수정 후 다시 배포:

```
```bash
# 1. 로컬 변경사항 커밋 & 푸시 (선택)
git add .
git commit -m "변경사항 설명"
git push origin main

# 2. 배포
npm run deploy
```
```



# 업데이트 후 배포



# 업데이트 후 배포

- Main page 업데이트

```
const Main = () => {
 return (
 <main className="main">
 <h1>컴퓨터 기기 쇼핑몰</h1>
 <section>
 <p>최신 컴퓨터 기기를 한 곳에서 만나보세요!</p>
 <div>

 </div>
 <div className="main-link">
 <Link to="/products">상품 목록 보기</Link>
 </div>
 </section>
 </main>
)
}
```



# 404 오류 문제

- 404 오류 – File Not Found

**문제:** 새로고침 시 `/product_shop/products` 같은 경로로 직접 접근하면 GitHub Pages가 해당 파일을 찾으려고 하는데, 실제 파일이 없어서 404가 발생합니다.

**해결방법:** `public/404.html`을 생성하여 `index.html`과 동일한 내용으로 설정하면, GitHub Pages가 모든 존재하지 않는 경로를 자동으로 `index.html`로 라우팅합니다.



# 404 오류 문제

- 404.html – 아직 오류 미해결

```
<!doctype html>
<html lang="en">
 <head>
 <meta charset="UTF-8" />
 <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <title>router</title>
 </head>
 <body>
 <div id="root"></div>
 <script type="module" src="./src/main.jsx"></script>
 </body>
</html>
```

문제는 [404.html](#)의 스크립트 경로입니다. Vite 빌드 후 [404.html](#)은 단순 복사되므로, [main.jsx](#) 경로가 무효입니다.

가장 간단한 해결책은 비트 빌드 후 자동으로 [index.html](#)을 [404.html](#)로 복사하도록 설정하는 것입니다:



# 404 오류 문제

## ▪ vite.config.js 설정 변경

```
// https://vite.dev/config/
export default defineConfig({
 plugins: [
 react(),
 {
 name: 'copy-404',
 writeBundle() {
 // 빌드 후 dist/index.html을 dist/404.html로 복사
 const distPath = path.resolve(__dirname, 'dist')
 const indexPath = path.join(distPath, 'index.html')
 const notFoundPath = path.join(distPath, '404.html')
 if (fs.existsSync(indexPath)) {
 fs.copyFileSync(indexPath, notFoundPath)
 }
 }
 }
],
 base: '/product_shop/'
})
```

추가

